Shaheed Zulfikar Ali Bhutto Institute of Science & Technology

COMPUTER SCIENCE DEPARTMENT

Total Marks: 7.5

Obtained Marks:

Artificial Neural Network (LAB)

Project Report

Title: SMART PLANT WATERING PREDICTION USING ARTIFICIAL NEURAL NETWORK

Submitted To: Mr. Hassan Mujtaba

Student Name: Syeda Nalain Zainab

Reg. Number: 24108233

SMART PLANT WATERING PREDICTION USING ARTIFICIAL NEURAL NETWORK

Abstract :

This project uses an Artificial Neural Network (ANN) to predict the watering Status (ON or OFF) from the TARP.csv dataset, which contains 100,000 records. The model uses four inputs: Soil Moisture, Temperature, Soil Humidity, and Time. The data was divided into training and testing sets, and standardised before training. The ANN has two hidden layers with 32 and 16 neurons and uses the Adam optimiser and early stopping. The model achieved 96.43% accuracy on 20,000 test samples. A simple Flask web page allows users to enter the four values and view the predicted status and confidence score. The project uses recorded data and does not control physical watering equipment. Further work can improve the consistency of preprocessing between training and the web interface.

1. INTRODUCTION

1.1 Background and Problem Statement

Plant watering depends on factors such as soil moisture, temperature, humidity, and time. Deciding when to water manually can be difficult. This project uses recorded data to predict the ON/OFF Status using an Artificial Neural Network (ANN).

An ANN learns patterns from examples and uses them to make predictions. In this project, the model is trained on TARP.csv. A simple Flask web page allows users to enter four values and receive a predicted status with a confidence score.

1.2 Objectives

The main objectives of this project are to:

- Examine the TARP.csv dataset, including missing values and Status distribution.
- Prepare the data and select the input features.
- Train an ANN using TensorFlow/Keras.
- Evaluate the model using accuracy, precision, recall, F1-score, and a confusion matrix.
- Save the trained model and preprocessing objects.
- Create a Flask web page to display predictions.

1.3 Scope and Related Work

This project focuses on training and testing an ANN in Google Colab and creating a simple Flask prediction interface. It does not include physical sensors, real-time data collection, IoT connectivity, or automatic pump control.

Previous studies have explored machine learning for irrigation management. Goap et al. (2018) studied an IoT-based irrigation system, while Adeyemi et al. (2018) used neural networks to model soil moisture for irrigation scheduling.

Compared with these systems, this project is a simple ANN-based study using recorded data. Its main purpose is to demonstrate how an ANN can predict watering status from selected input features.

This project uses an Artificial Neural Network (ANN) to predict the watering Status (ON or OFF) from the TARP.csv dataset, which contains 100,000 records. The model uses four inputs: Soil Moisture, Temperature, Soil Humidity, and Time. The data was divided into training and testing sets, and standardised before training. The ANN has two hidden layers with 32 and 16 neurons and uses the Adam optimiser and early stopping. The model achieved 96.43% accuracy on 20,000 test samples. A simple Flask web page allows users to enter the four values and view the predicted status and confidence score. The project uses recorded data and does not control physical watering equipment. Further work can improve the consistency of preprocessing between training and the web interface.

2. DATASET AND PREPROCESSING

2.1 Dataset Description

The TARP.csv dataset contains 100,000 records and 15 columns. Four complete columns were selected as model inputs: Soil Moisture, Temperature, Soil Humidity, and Time.

The other ten numeric columns contain many missing values, so they were not used. Five columns have approximately 76% missing values, while the other five have approximately 97.8% missing values. The selected four input features have no missing values.

The original column name contains a leading space before “Soil Humidity,” which is preserved in the saved feature list. The dataset's source, collection method, units, and the exact meaning of Time are not documented in the available project materials.

Dataset shape (rows, columns): (100000, 15)

	Soil Moisture	Temperature	Soil Humidity	Time	Air temperature (C)	Wind speed (Km/h)	Air humidity (%)	Wind gust (Km/h)	Pressure (KPa)	ph	rainfall	N	P	K	Status
0	54	22	70	21	19.52	2.13	55.04	6.30	101.50	6.502985	202.935536	90.0	42.0	43.0	ON
1	12	20	40	104	19.49	2.01	55.17	10.46	101.50	7.038096	226.655537	85.0	58.0	41.0	OFF
2	34	26	35	62	19.47	1.90	55.30	14.63	101.51	7.840207	263.964248	60.0	55.0	44.0	ON
3	7	44	44	93	19.54	2.28	54.20	16.08	101.51	6.980401	242.864034	74.0	35.0	40.0	OFF
4	50	38	23	92	19.61	2.66	53.09	17.52	101.51	7.628473	262.717340	78.0	42.0	42.0	OFF

Figure 1: Dataset shape (100000, 15) and first five records of TARP.csv (notebook output).

Table 1: Columns of TARP.csv and their use in the model (n = 100,000 records).

Column(s)	Type	Missing	Use
Soil Moisture	int64	0%	Input feature
Temperature	int64	0%	Input feature
Soil Humidity	int64	0%	Input feature
Time	int64	0%	Input feature
Air temperature, wind speed, air humidity, wind gust, pressure (5 columns)	float64	76.0%	Not used (> 50% missing)
ph, rainfall, N, P, K (5 columns)	float64	97.8%	Not used (> 50% missing)
Status	text	0%	Target (OFF 46,350; ON 53,650)

2.2 Target Variable

The Status column contains two labels:

- OFF: 46,350 records (46.35%)
- ON: 53,650 records (53.65%)

The class distribution is slightly imbalanced. Therefore, a stratified split was used to keep similar class proportions in the training and test sets. The model learns to predict the Status labels recorded in the dataset. The exact real-world meaning of ON and OFF is not documented.

2.3 Preprocessing Workflow

The following steps were used to prepare the dataset for training:

- Target cleaning: Extra spaces were removed, and the labels were converted to uppercase. Both classes, OFF and ON, were checked for missing values.
- Feature selection: Four complete numeric features were selected: Soil Moisture, Temperature, Soil Humidity, and Time. Columns with many missing values were excluded.
- Label encoding: The text labels were converted into numbers: OFF = 0 and ON = 1. The class order was saved for interpreting predictions.
- Train–test split: The dataset was divided into 80,000 training records and 20,000 test records using an 80/20 stratified split with random_state = 42.
- Imputation: A median imputer was fitted on the training data. Since the selected features had no missing values, imputation did not change them.
- Standardisation: A StandardScaler was fitted on the training data and applied to both datasets. This helps put the input features on comparable scales and supports model training.

Table 2: Preprocessing pipeline as implemented.

Step	Method	Result
Target cleaning	Strip, upper-case, drop missing labels	No missing labels; classes OFF and ON
Feature selection	Keep numeric columns with ≤ 50% missing	4 of 14 numeric columns retained
Target encoding	scikit-learn LabelEncoder (alphabetical)	OFF = 0, ON = 1
Train–test split	Stratified, test_size = 0.20, random_state = 42	80,000 training / 20,000 test samples
Imputation	SimpleImputer (median), fitted on training data	No effect here (no missing values)
Scaling	StandardScaler, fitted on training data only	Zero mean, unit variance inputs

3. ANN METHODOLOGY

The model is a feed-forward Artificial Neural Network (ANN) built using Keras. It has four input features, two hidden layers with 32 and 16 neurons, and an output layer with two neurons to predict OFF or ON.

The first two layers use ReLU activation, and the output layer uses softmax to calculate the probabilities of both classes. The model has **722 trainable parameters**. Dropout (0.2) is used to help reduce overfitting.

The model uses the **Adam optimiser** and sparse categorical cross-entropy loss. Training was performed with a batch size of 32 for up to 50 epochs.

For validation, 20% of the training data (16,000 samples) was set aside, leaving 64,000 samples for training. Early stopping monitored validation loss and restored the best model weights. The random seeds were set to 42 to support reproducibility.

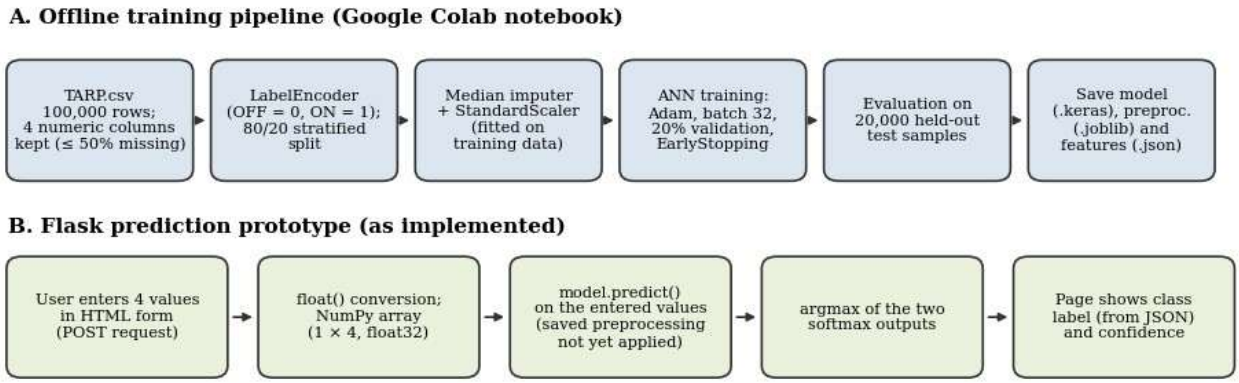
The 20,000 test samples were kept separate from training and validation and used for the final evaluation of the model.

Table 3: ANN architecture and training configuration.

Layer	Configuration	Output shape	Params	Training setting
Input	4 standardised features	(None, 4)	0	Optimiser: Adam (lr = 0.001)
Dense 1	32 units, ReLU	(None, 32)	160	Loss: sparse categorical cross-entropy
Dropout	rate 0.20	(None, 32)	0	Metric: accuracy; batch size 32
Dense 2	16 units, ReLU	(None, 16)	528	Max. 50 epochs; validation split 0.20
Output	2 units, softmax	(None, 2)	34	EarlyStopping: val_loss, patience 5, best weights
Total			**722**	**Random seed 42**

4. SYSTEM IMPLEMENTATION

Both parts of the system were implemented in a single Google Colab notebook (Python 3.13; Figure 2). The training pipeline uses pandas and NumPy for data handling, scikit-learn (Pedregosa et al., 2011) for encoding, splitting, imputation, scaling and metrics, TensorFlow/Keras (Abadi et al., 2016) for the network, and Matplotlib for plots. After evaluation, the notebook saves three artefacts: the trained network (watering_model.keras), a joblib dictionary holding the fitted imputer, scaler, label encoder and feature list (watering_preprocessing.joblib), and a JSON file with the target name, feature order and class names (watering_features.json).



Workflow of the training pipeline (A) and the Flask prediction prototype (B), drawn from the implemented code.

6.2 Flask Application

The Flask application uses one route, "/", for both GET and POST requests. The web form takes four inputs in the same order used during training: Soil Moisture, Temperature, Soil Humidity, and Time.

When the user submits the values, they are converted into numbers and given to the trained ANN model. The model predicts either ON or OFF and shows the probability of the predicted class as the confidence score.

Errors are displayed as messages. The Flask server runs in Google Colab and is accessed through a temporary ngrok link. The application only provides model predictions; it does not directly control watering hardware.

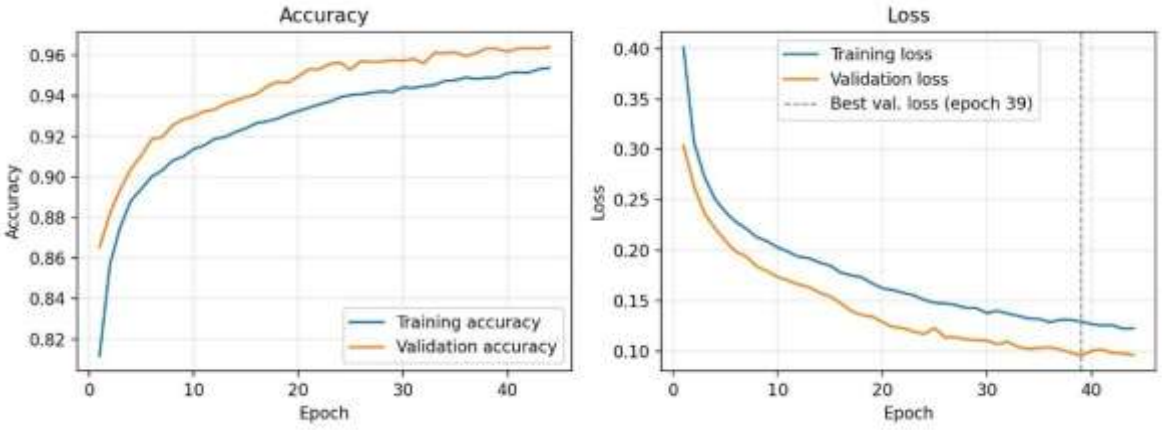
5. RESULTS AND EVALUATION

5.1 Training and Validation Performance

Training ran for 44 epochs. The lowest validation loss (0.0956) occurred at epoch 39; five further epochs brought no improvement, so early stopping ended training at epoch 44 and the weights of epoch 39 were restored. Table 4 keeps training, validation and test performance separate. Keras reports training metrics as running averages with dropout active, which is why validation accuracy can exceed training accuracy. Figure 3 shows both accuracy curves rising and both loss curves falling steadily before levelling off with a small gap, which indicates stable learning without marked over-fitting.

Table 4: Training, validation and test performance (restored epoch-39 weights).

Data subset (samples)	Accuracy	Loss	Source
Training (64,000), epoch 39	0.9490	0.1296	Training log
Validation (16,000), epoch 39	0.9633	0.0956	Training log
Test (20,000), held out	0.9643	0.0945	Test evaluation



- **Training and validation accuracy (left) and loss (right) per epoch, re-plotted from the logged epoch values.**

5.2 Test Accuracy and Classification Metrics

On the 20,000 held-out test samples the model reached 96.43% accuracy with a test loss of 0.0945, close to the validation figures in Table 4. Table 5 gives the per-class metrics. Precision is 0.97 for OFF and 0.96 for ON, and

recall is 0.95 for OFF and 0.98 for ON, giving F1-scores of 0.96 and 0.97. The macro and weighted averages are both about 0.96, so performance is balanced across the two classes despite the mild class imbalance.

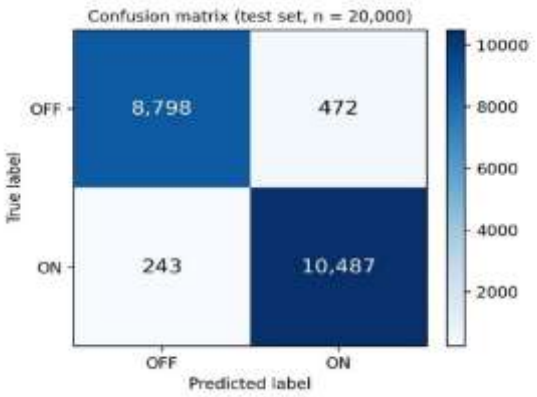
Table 5: Classification report on the test set.

Class	Precision	Recall	F1-score	Support
OFF	0.97	0.95	0.96	9,270
ON	0.96	0.98	0.97	10,730
Accuracy			0.96	20,000
Macro avg.	0.97	0.96	0.96	20,000
Weighted avg.	0.96	0.96	0.96	20,000

5.3 Confusion Matrix

The image shows the confusion matrix for the test set. Out of 9,270 OFF samples, 8,798 were correctly predicted, while 472 were classified as ON. Out of 10,730 ON samples, 10,487 were correctly predicted, while 243 were classified as OFF.

Overall, the model made 715 incorrect predictions, representing 3.58% of the test set. It performed slightly better in identifying ON samples than OFF samples. These results show that the model predicted both classes with good accuracy.



6. FLASK INTERFACE AND DISCUSSION

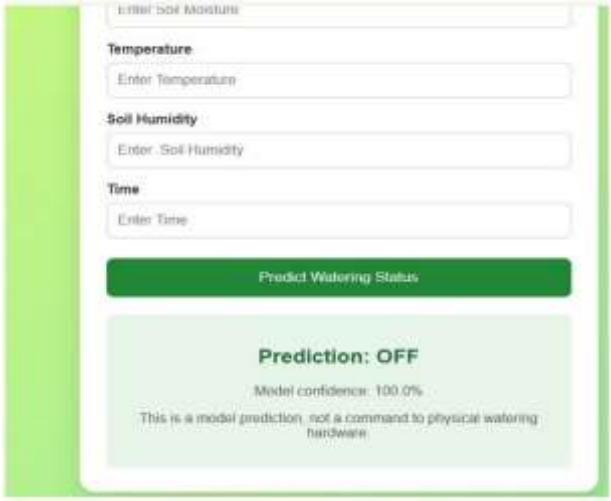
6.1 Interface and Sample Prediction

The images show the Flask input form and its prediction result. The interface accepts four inputs: Soil Moisture, Temperature, Soil Humidity, and Time. After clicking the “Predict Watering Status” button, it displays the predicted status and confidence score.

In the example, the model predicted OFF with 100% confidence. This result applies only to the given input and does not represent the model’s overall accuracy. The interface demonstrates predictions only and does not control physical watering hardware.

- **Flask input form and Flask prediction result (one example).**





6.3 Discussion:

The ANN achieved 96.43% test accuracy, showing good performance in predicting the recorded ON/OFF Status labels. The results are based on one training run and one test split. The Flask interface also successfully demonstrates how users can enter values and view predictions. Further testing with different data and comparison with other models could help evaluate the system more thoroughly

.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

This project developed an Artificial Neural Network (ANN) to predict the ON/OFF Status in the TARP.csv dataset using four inputs: Soil Moisture, Temperature, Soil Humidity, and Time. The model achieved **96.43% accuracy on 20,000 test samples**. A Flask web interface was also developed to display predictions and confidence scores. Overall, the project demonstrates the use of ANN for plant-watering status prediction.

7.2 Future Work

In the future, the project can be improved by:

1. Ensuring the same preprocessing steps are used in the Flask interface.
2. Adding more information about the dataset and its features.
3. Testing the model with different data to evaluate its performance.
4. Improving the web interface for more reliable use.
5. Exploring real sensors for smart plant watering.

REFERENCES

Abadi, M. et al. (2016). **TensorFlow:** A system for large-scale machine learning. *OSDI '16*.

Adeyemi, O. et al. (2018). Dynamic neural network modelling of soil moisture content for predictive irrigation scheduling. *Sensors, 18*(10).

Goap, A. et al. (2018). An IoT based smart irrigation management system using machine learning and open source technologies. *Computers and Electronics in Agriculture, 155*.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *ICLR 2015*.

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature, 521*.

Pedregosa, F. et al. (2011**). Scikit-learn**: Machine learning in Python. *Journal of Machine Learning Research, 12*.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature, 323*.

Srivastava, N. et al. (2014). **Dropout**: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research, 15*.

Dataset: TARP.csv

Dataset source : Kaggle